QUBIX

Admin, Operations & Disaster-Recovery Guide

The operator-grade reference for running Qubix in normal times and in an incident — deployment, data model, backups, and emergency playbooks.

Internal codename **Strata** · public brand **Qubix** · operated by **Arcave Technologies**
Production reference `2f3238f` · deployed 2026-07-22 · `strata-nine-pi.vercel.app`
Document version 2.0 · generated 2026-07-22

VERIFIED confirmed from code/system **UNVERIFIED** needs admin access **PROPOSED** recommended, not yet in place **REQUIRES APPROVAL** Ali must authorise

Contents

A · Authority, ownership & document control

Document control

Field	Value
Document owner	Ali (founder / primary operator) REQUIRES APPROVAL to change ownership
Authoring / technical reviewer	Claude (drafting) & Codex (cross-review), under Ali's direction
Last code verification	2026-07-22, against `origin/main = origin/staging = 2f3238f`
Last live verification	2026-07-22 — homepage & legal pages HTTP 200; security headers verified post-deploy by Codex under Ali's authorisation
Production commit / deploy	`2f3238f` / Vercel `dpl_84E64sJmRPzABzm9LpDDPg1Tp8rq`
Next review due	On the next production deploy, or 2026-10-22, whichever is first

Revision history. v1.0 (2026-07-22) — first 14-page guide. v2.0 (2026-07-22) — operator-grade rewrite: corrected release/DB/server facts, added full schema/API appendix, disaster recovery, 12 incident playbooks, secret-rotation, CSP completion, support/privacy workflows, one-page emergency card, verification labels. v2.1 (2026-07-22) — Codex cross-review corrections: split fresh-bootstrap from the 0006 repair (0006 not applied to a fresh DB); PowerShell-first destructive-test commands; corrected live-verification attribution (Codex under Ali's authorisation); removed unverified provider-plan backup claims; PDF/UA tagging where supported.

Precedence of sources

When two sources disagree, trust them in this order. The running system always wins.

1. **The running production system** — what the live site and Supabase project actually do.
2. `docs/PUBLIC-BETA-CHECKLIST.md` — authoritative live status of every gate.
3. `docs/ENVIRONMENTS.md` and `docs/RELEASE-MODEL.md` — configuration & release model.
4. **The migration files** in `supabase/migrations/` — the schema of record.
5. **This guide.** If it conflicts with any of the above, the above wins and this guide is stale.

Access & ownership matrix

Roles and recovery paths only — **never** record credentials here. UNVERIFIED rows need Ali to confirm the owning account and that recovery (2FA backup codes, recovery email) is in place.

System	Owner / account	Recovery path to confirm
GitHub (`alisid0/strata`)	Ali **UNVERIFIED**	Account 2FA backup codes; is anyone else an admin?
Vercel (production project)	Ali / `qubixprod` team **UNVERIFIED**	Team recovery; SSO vs password; 2FA
Supabase Production (`wmetdmfsniqrshuaoodc`)	Ali **UNVERIFIED**	Org owner; MFA; is the DB password stored safely?
Supabase Staging (`atmmfkhjsdqqwnhqifxm`)	Ali **UNVERIFIED**	Same org as production?
DNS / registrar (`arcavetech.co.uk`)	Ali **UNVERIFIED**	Registrar login + 2FA; who can change records
Google OAuth (Cloud Console)	Ali **UNVERIFIED**	Project owner; consent-screen contact
SMTP provider	Ali **UNVERIFIED**	Provider account; sending-domain DNS
Google Play (dev acct `6992886775198578677`)	Arcave Technologies	Play Console access; **upload/signing key backup**
Google Search Console	Ali **UNVERIFIED**	Verified via DNS TXT — **do not delete that record**
Support mailbox `admin@arcavetech.co.uk`	Ali **UNVERIFIED**	Who monitors it; forwarding; retention

Approval gates — actions that require Ali's explicit go-ahead

> None of the following may be done by an agent or operator without Ali approving first, in writing on the board or equivalent:
>
> - Any **production deploy** (`pnpm run deploy`).
> - Any **schema or RLS change** applied to the production database.
> - Any **DNS or custom-domain move**.
> - Any **secret rotation** or credential change.
> - Any **destructive user-data action** (bulk delete, manual account deletion).
> - Flipping the **CSP from report-only to enforcing**.
> - Running any **destructive test against anything but staging**.

B · Architecture & complete data/API reference

Runtime components & trust boundaries

Qubix has **no always-on custom application server**. It does, however, run server-side code in two managed places: Supabase **Edge Functions** and one **dormant** Vercel serverless function. The distinction matters for incident response.

Component	Runs where	Credential it may use	Trust boundary
Svelte SPA	User's browser / Android TWA, served from Vercel CDN	Public anon key only	Untrusted client — RLS enforces access
Supabase Auth	Supabase (managed)	—	Owns identities; app never sees passwords
PostgreSQL + RLS	Supabase (managed, London)	Enforces per-user rows	The security boundary of record
Storage buckets	Supabase (managed)	Public read (2) / per-user (1)	Screenshots private per folder
`delete-account` Edge Function	Supabase Deno runtime	**Service-role key** (injected)	Trusted server code; the only place the service key is used at runtime
`api/csp-report.js`	Vercel serverless — **dormant**	None	Deployed but unreferenced; 204s until wired
Ingest / admin scripts	Operator's machine, on demand	**Service-role key** from local env	Trusted, human-run; never in CI or browser

Every Supabase call site VERIFIED

Complete inventory from the codebase at `2f3238f`. There is one client, in `src/lib/supabase.js`. Every write carries an explicit `user_id` and is enforced by RLS.

File	Operation	Target	Auth assumption / control
`lib/stores/auth.js`	signUp, signInWithPassword, signInWithOAuth, signInWithOtp, verifyOtp, resetPasswordForEmail, updateUser, resend, signOut, getSession, onAuthStateChange	Supabase Auth	Public flows; OTP path gated off
`lib/stores/profile.js`	select / upsert	`user_profiles`	Own row (RLS)
`lib/stores/progress.js`	select ×5, upsert ×3, insert ×2	`user_board_progress`, `user_path_progress`, `user_quiz_attempts`, `user_workshop_attempts`, `user_w_events`, `user_daily_activity`	Own rows (RLS); reads also filter `eq('user_id')`
`lib/stores/engagement.js`	insert	`user_engagement_sessions`	Own row (RLS)
`lib/stores/issueReports.js`	insert; storage upload	`issue_reports`; `issue-screenshots`	Signed-in only (F-04); upload under own folder
`lib/supabase.js`	rpc	`export_my_user_data()`	Authenticated; self-scoped
`lib/supabase.js`	fetch POST	`functions/v1/delete-account`	Bearer JWT + confirm phrase
`lib/content/dynamicBoards.js`	select	`cards`	Public read

Admin-only script call sites (service role, never shipped to users): `scripts/ingest-bbs.mjs`, `ingest-snippets.mjs`, `ingest-final-review.mjs`, `sync-bbs-from-md.mjs`, `migrate-dynamic-kickers.mjs`, `generate-audio.mjs`, `progress-report.mjs` — all write `cards` or storage.

Full schema — every table VERIFIED

From `schema.sql` + migrations 0002–0007. All `user_*` tables: RLS enabled, `user_id` → `auth.users(id)` **ON DELETE CASCADE**, own-row policies for select/insert/update/delete. Grants set explicitly in 0007.

cards — public catalogue (service-role write only)

Column	Type	Constraints
id	uuid	PK, default uuid_generate_v4()
sort_order	integer	not null, **unique** — the BB number
act	text	not null, check in I–V
kicker, title	text	not null
layers	jsonb	not null default '[]' — floor HTML
img_url	text	nullable
tags	jsonb	nullable
created_at, updated_at	timestamptz	default now(); updated_at via trigger

Index: `cards_sort_order_idx`. RLS: public read; no anon/authenticated write policy. Grant (0007): `select` to anon + authenticated.

user_profiles — one private profile per account

Column	Type	Constraints
user_id	uuid	PK → auth.users, ON DELETE CASCADE
internal_username	text	not null, unique, regex `^[a-z0-9_]{3,40}$`; from user-id not email (F-03)
age_band	text	check: 13_15 / 16_17 / 18_plus / prefer_not (under_13 removed, F-08)
learning_goal	text	check: exam/school/career/curiosity/coding/teaching/prefer_not
daily_goal_minutes	integer	check in (5,10,15,30)
selected_topics	jsonb	not null default '[]'
heard_from, learner_type	text	nullable
onboarding_completed	boolean	not null default false
created_at, updated_at	timestamptz	default now()

user_board_progress (PK user_id+bbid)

bbid int; first/last_opened_at, open_count (≥0), deepest_floor_reached (≥0), deepest_floor_completed_at, recall_due_at, recall_stage (≥0), recall_passes (≥0), created/updated_at. Indexes on (user_id,last_opened_at) and partial (user_id,recall_due_at).

user_path_progress (PK user_id+path_id)

path_id text; first/last_opened_at, best_score, best_total, first_pass_at, mastered_once_at, recalled_mastered_twice_at, timestamps. Check: score valid vs total.

user_quiz_attempts (PK id uuid) / user_workshop_attempts (PK id uuid)

quiz: path_id, score(≥0), total(>0), completed_at, metadata; check score≤total. workshop: module_id, score, total, best_streak(≥0), is_challenge bool, completed_at, metadata.

user_w_events (PK id uuid)

event_type, event_ref, amount(>0), bonus bool, repeatable bool, created_at, metadata. Unique partial index (user_id,event_type,event_ref) where not repeatable.

user_daily_activity (PK user_id+activity_date) / user_engagement_sessions (PK id uuid)

daily: activity_date, event_count(≥0), timestamps. sessions: started_at, ended_at (≥started), active_seconds(≥0), route, metadata, created_at.

issue_reports (PK id uuid)

Column	Notes
user_id	→ auth.users **ON DELETE SET NULL** (report survives, anonymised)
category	check: bug/content/workshop/audio/visual/account/idea/other
message	not null, 3–4000 chars
route, bbid, workshop_id, screenshot_path, app_version, user_agent	context (cleared on deletion — F-06)
status	check: open/reviewing/fixed/closed; user may only set 'closed' (F-07 trigger)
viewport, metadata	jsonb; cleared on deletion (F-06)

Insert: authenticated only, `auth.uid()=user_id` (F-04). Update guarded by `guard_issue_report_update()`; insert rate-limited by `guard_issue_report_rate()` (10/hr). Legacy `public.progress` retired in 0005 (F-09).

Functions, triggers, buckets, Edge/serverless VERIFIED

Object	Type	Privilege & behaviour
`create_private_user_profile()`	trigger on auth.users insert	security definer; makes neutral-username profile
`delete_my_user_data()`	RPC	security definer; erases caller's learning data, anonymises reports; execute→authenticated only (0007)
`export_my_user_data()`	RPC	returns caller's data as JSON; execute→authenticated only (0007)
`guard_issue_report_update()`	trigger	blocks edits after submit; allows deletion-time anonymisation
`guard_issue_report_rate()`	trigger	10 reports/user/hour
`set_updated_at()`	trigger	maintains updated_at
`card-images`, `card-audio`	storage bucket	public read
`issue-screenshots`	storage bucket	private; policy scopes each user to `<uid>/`
`delete-account`	Supabase Edge Function	service-role; verifies JWT + confirm phrase; drains screenshots, calls RPC, deletes identity; fails loud if identity delete fails (orphan flagged)
`api/csp-report.js`	Vercel serverless	dormant; POST-only, trims to path, drops noise, never logs IP

Migration & bootstrap order VERIFIED

Two distinct sequences. Per `docs/ENVIRONMENTS.md`, a fresh clean bootstrap is **schema → 0002 → 0003 → 0004 → 0005 → 0007** (plus the Edge Function). **0006 is NOT part of a fresh bootstrap** — it is an incremental repair for a database that already had the earlier broken 0005.

Fresh bootstrap (rebuild from scratch)

Step	File	Role
1	`supabase/schema.sql`	Base: cards, legacy progress, first policies, set_updated_at
2	`0002_storage_buckets.sql`	The 3 buckets
3	`0003_bbid.sql`	Board-id handling
4	`0004_secure_user_data.sql`	User-data model: all user_* tables, RLS, profile trigger
5	`0005_launch_hardening.sql`	F-03,04,06,07,08,09; export fn; defines `delete_my_user_data()`
6	`0007_explicit_api_grants.sql`	Explicit role grants; revoke default execute
7	deploy `delete-account` Edge Fn + set its `SUPABASE_SERVICE_ROLE_KEY` secret	The Edge Function removes screenshots via the Storage API

> **Do NOT apply `0006` on a fresh bootstrap.** On a clean build the corrected `delete_my_user_data()` is already in place from 0005 as combined with the Edge Function, and screenshot deletion is handled by the Edge Function — not by SQL. Applying 0006 to a fresh DB is unnecessary and misleading about where the fix lives.

Conditional upgrade / repair — earlier-0005 databases only

File	When to apply
`0006_delete_account_storage_fix.sql`	**Only** to a database that had the earlier 0005 whose `delete_my_user_data()` tried to delete from `storage.objects` directly (which Supabase rejects). It replaces that function with the corrected version. Not needed on a fresh bootstrap.

> **VERIFIED** against migration contents and `docs/ENVIRONMENTS.md`. Before any from-scratch rebuild, reconcile against the Supabase project's actual migration history in the dashboard.

C · Normal release runbook

REQUIRES APPROVAL The production deploy step needs Ali's go-ahead.

Preconditions

1. `git fetch origin` — start from the true remote state, not a stale local clone.
2. Identify the exact reviewed commit to ship; confirm `origin/staging` is what was reviewed.
3. Clean, isolated worktree — no unrelated modified or untracked files. In particular confirm no stray public asset (e.g. a rejected image) is staged: `git status` should be clean.
4. Secret scan: no key material in the diff.
5. Security tests green: `pnpm run test:security` (47/47).
6. Staging QA passed on `qubix-staging.vercel.app`.
7. Production build succeeds: `pnpm run build:production`.

Git checks & stop conditions

```
git fetch origin
git log --oneline origin/main..origin/staging   # what will ship
git rev-parse origin/main origin/staging         # capture both hashes
```

> **STOP** if either remote moved since review, if `origin/main` is not an ancestor of `origin/staging` (not a clean fast-forward), or if a local `main` is polluted (a prior incident put a staging commit on a local `main`). Recover `main` from `origin/main`; never deploy from an unreviewed local branch.

Deploy & deployment-ID capture

```
pnpm run build:production
pnpm run deploy                # manual — this changes the live site
# record the Vercel deployment ID (e.g. dpl_...) and confirm the stable
# production alias points at it.
```

Confirm the deployed commit matches Git: the bundle should contain the production Supabase ref and not the retired ref.

Post-deploy smoke test

Check	Pass criteria
Homepage + `/privacy.html` + `/terms.html`	HTTP 200
Sign-in (email & Google)	Completes, session persists
Board loads incl. a dynamic card (85+)	Renders sanitised HTML
Progress write	A board open persists for a test account
Logout / session restore	Clean, where safe to test
Service worker	New build activates (skipWaiting)
Asset URLs	Content-hashed, cache-immutable
Console / network	No severe errors, no failed requests
Security headers	See below

Expected headers now **verified post-deploy 2026-07-22 by Codex under Ali's authorisation**: CSP *report-only* present; enforcing CSP **absent** (by design); HSTS; `X-Frame-Options: DENY`; `X-Content-Type-Options: nosniff`; `Referrer-Policy: strict-origin-when-cross-origin`.

Go / No-Go

Evidence field	Record
Source commit	_____
Deployment ID	_____
Smoke test	pass / fail
Headers present	yes / no
Decision	GO / NO-GO

Rollback threshold: any smoke-test failure that affects sign-in, board loading, or data writes → roll back immediately (§E / Playbook 1), do not "fix forward" under pressure.

D · Database & content-change runbooks

Additive-first schema change

> **REQUIRES APPROVAL** Applying to production is irreversible and immediate. Git and Vercel do **not** promote schema; a migration file in Git is not "applied" until run in the SQL editor.

1. Confirm backup/restore readiness (§E) before any production DDL.
2. Write the change as a new numbered migration (`0008_*.sql`). Prefer additive: add a column/table, backfill, then stop reading the old shape in a later deploy.
3. Apply in the **staging** SQL editor; run both isolation tests (below); QA the app.
4. Deploy tolerant client code to production **first**.
5. Apply the migration in the **production** SQL editor.
6. Verify immediately: RLS still isolates, grants intact, no user-facing breakage.
7. Keep a corrective-migration plan ready (there is no automatic rollback).

Running the destructive staging tests safely

Both scripts create and delete accounts. They refuse anything but staging via a four-gate fail-closed guard. Provide the staging service-role key in-shell for the run only — **never** write it to a file or into this guide.

Windows PowerShell (Ali's environment)

```
$env:SUPABASE_SERVICE_ROLE_KEY = '<staging service role key>'
node --env-file=.env.staging.local scripts/test-rls-isolation.mjs --staging
node --env-file=.env.staging.local scripts/test-user-data-lifecycle.mjs --staging
Remove-Item Env:SUPABASE_SERVICE_ROLE_KEY
```

Bash / macOS / Linux (equivalent)

```
export SUPABASE_SERVICE_ROLE_KEY='<staging service role key>'
node --env-file=.env.staging.local scripts/test-rls-isolation.mjs --staging
node --env-file=.env.staging.local scripts/test-user-data-lifecycle.mjs --staging
unset SUPABASE_SERVICE_ROLE_KEY
```

Guard	Behaviour
No `--staging`	Refuse, exit 2
Target host = production	Refuse (host match)
Target host ≠ configured staging host	Refuse
Correct staging host	Proceed

> **Absolute ban:** never point these at production, never set an `RLS_ALLOW_PROD`-style override. Verify production isolation with read-only checks only. Expected result: all checks pass; both scripts clean up their test accounts in a `finally` block. Confirm no `rls-test` / `lifecycle-` accounts remain afterward.

Content export/import & card rollback

Content lives in the Supabase `cards` table, not Git. A code deploy never changes content; a card edit never needs a deploy.

- **Edit a card:** change the row in the Supabase table editor (or ingest scripts with the service-role key). Live on the learner's next open of that board.

- **Refresh staging from production catalogue:** `pnpm run content:export-staging-sql` → review the generated SQL chunks → apply in staging → verify the card count. Public cards only; never copies users or private data.
- **Rollback a bad edit:** edit the row back to its previous value. Keep a note of the prior content before a bulk change.

E · Backup, restore & disaster recovery

> Several capabilities below are **UNVERIFIED** because confirming them needs Supabase/Vercel/ registrar dashboard access I do not have. Each gives the exact admin check. Treat an unverified backup as **no backup** until confirmed.

Capability status by system

System	Capability	Status	Admin check / gap
Supabase Postgres	Automated backups / Point-in-Time Recovery	**UNVERIFIED**	Dashboard → Database → Backups: record the plan, backup frequency, retention window, PITR availability/ status, last successful backup, and the restore procedure. (Provider plan features change — do not assume; check and date the finding.)
Supabase schema/ functions/ policies	Reproducible from Git migrations	**VERIFIED**	Migrations 0002–0007 + schema.sql rebuild it
Public catalogue (cards)	Exportable via script	**VERIFIED**	`content:export-staging-sql` / catalogue audit
Storage objects	Backup	**UNVERIFIED**	Not covered by DB backup. Decide: are card-images/audio reproducible from source, and are user screenshots worth backing up? (They are deletable personal data.)
Auth users	Backup/export	**UNVERIFIED**	Auth export is limited. Never copy production auth users to staging.
Vercel deployment	Previous-deployment rollback	**VERIFIED (mechanism)**	Immutable prior builds selectable in dashboard
Git history	Reviewed remote commits	**VERIFIED**	Recover from `origin`, not polluted local branches
DNS / domain	Registrar-held	**UNVERIFIED**	Confirm registrar access + record backup
Android signing/ upload key	Backed up	**UNVERIFIED**	Once a key exists, its loss is unrecoverable except via Play App Signing key reset. Back it up offline.

Restore procedure (database)

> Restore to a **separate recovery project first**. Never make production the first target of a restore test.

1. Create a fresh recovery Supabase project.
2. Restore the backup / apply migrations in bootstrap order (§B).
3. Verify: table integrity, RLS enabled + policies present, grants (0007), auth callback URLs, storage buckets, card count and sample content.
4. Point a staging build at the recovery project and smoke-test.

5. Only after verification, plan the production cutover (auth Site URL, redirect allowlist, client env, DNS if needed).

Other recovery paths

- **Vercel:** select the previous good deployment, reassign the stable production alias, capture the new deployment ID, re-run the smoke test.
- **Git:** reset local `main` to `origin/main`; never rely on a local branch that may carry stray commits.
- **DNS/domain move:** keep the Search Console TXT record; verify HTTPS/cert issues after the move; account for TTL; update Supabase auth Site URL and OAuth redirect URIs to the new origin.
- **Edge Function / secrets:** redeploy `delete-account`; re-set `SUPABASE_SERVICE_ROLE_KEY` as a function secret. Secrets are not in Git — keep an offline record of *which* secrets exist (not their values).
- **Android:** the upload key and Play App Signing boundary determine recoverability; document where the keystore is backed up.

RPO / RTO — proposed targets vs verified capability

	Proposed target	Verified capability
RPO (max data loss)	PROPOSED ≤ 24h for user data	UNVERIFIED — depends on Supabase backup tier
RTO (time to restore)	PROPOSED ≤ 4h for the web app; DB per provider	Vercel rollback: minutes. DB: UNVERIFIED

Do not claim an SLA or a backup feature that has not been checked in the dashboard.

Restore-drill template PROPOSED

Schedule a restore drill (e.g. quarterly). Record: date, operator, backup source + timestamp, recovery project ref, integrity/RLS/grant/auth/storage/card-count results, time taken (actual RTO), data-loss window (actual RPO), issues found, sign-off.

F · Emergency incident playbooks

Each: **first 15 minutes → containment → recovery → verification → communication → follow-up**. Actions marked APPROVAL need Ali first. **Never**, as a "quick fix": disable RLS, expose a service key, copy real users to staging, delete evidence, or apply an unreviewed production migration.

1 · Public site down / bad Vercel deployment

15 min: Confirm scope — is it the site or the whole of Vercel? Check the Vercel dashboard build/function logs and the deployment status.

Containment: Roll back — reassign the stable production alias to the last known-good deployment ID. This is safe and immediate.

Recovery: Capture the new (old) deployment ID; investigate the bad build offline.

Verify: Homepage + legal 200; sign-in; a board loads; headers present.

Comms: If users were affected, note duration and cause for the record.

Follow-up: Fix forward on a branch → staging → re-deploy per §C.

2 · Broken auth / SMTP / recovery links / Google OAuth

15 min: Reproduce. Which flow — sign-up email, password reset, or Google? Check Supabase Auth logs.

Containment: If a recent change caused it, roll the client back (Playbook 1). Do not disable auth.

Recovery: SMTP → check provider status + sending-domain SPF/DKIM. OAuth → the redirect URI must exactly equal the Supabase callback `https://wmetdmfsniqrshuaoodc.supabase.co/auth/v1/callback`. Recovery links to localhost/staging → auth **Site URL** is wrong for the environment.

Verify: Full loop on a real device — sign-up, confirm, reset, Google, logout.

Follow-up: Record which config was wrong; update the auth runbook.

3 · Suspected secret leak (service-role / Vercel / SMTP / OAuth / GitHub / DNS / signing key)

15 min: Treat as live. Identify which secret and where it may have leaked (commit, log, screenshot). Preserve evidence — do **not** delete the leak; screenshot/record it without copying the secret value anywhere new.

Containment APPROVAL**:** Rotate the affected secret at its source (§G rotation order). The service-role key is the highest priority — its leak exposes all data.

Recovery: Re-set the new secret everywhere it is consumed (dashboard, Edge Function secret, local env). Redeploy/redeploy functions as needed.

Verify: Old secret no longer works; app still functions; deletion/export still work if the service key rotated.

Follow-up: Assess exposure window; if personal data may have been accessed, escalate to Ali for a breach decision (do not invent legal deadlines).

4 · Suspected cross-user exposure / RLS regression

15 min: Capture the report/evidence. Identify the table and operation.

Containment: Do **not** disable RLS. If a recent migration caused it, prepare a corrective migration; if a recent client change, roll it back.

Recovery APPROVAL**:** Apply a corrective migration (staging-tested) restoring the own-row policy.

Verify: Run `test-rls-isolation.mjs` against staging (never prod); read-only checks against production.

Follow-up: Add a regression test; breach assessment with Ali.

5 · Accidental production schema/RLS change

15 min: Stop further changes. Identify exactly what was run and when (SQL editor history).

Containment: Assess blast radius — did it drop/alter a column an active client reads? If so, expect running sessions to break.

Recovery APPROVAL**:** Write and staging-test a corrective migration. If data was lost, consider a restore to a recovery project (§E) to recover rows. No automatic undo exists.

Verify: Schema matches intent; RLS/grants intact; app works.

Follow-up: Reinforce the approval gate; never run ad-hoc DDL on production.

6 · Public catalogue content deleted/corrupted

15 min: Determine which cards and how (bad ingest run? manual edit?). Halt any running ingest script.

Containment: No user data is at risk — this is public content. Note the current row count vs the expected 1,145.

Recovery: Re-import from source (`1945_BBs/`, content drafts) via the ingest scripts, or restore `cards` from a DB backup to a recovery project and copy rows back. Public-card-only.

Verify: Count = expected; spot-check sample boards render.

Follow-up: Add a pre-ingest count/backup step.

7 · Malicious/unsafe HTML content or XSS report

15 min: The render-time sanitiser (F-01) is the active defence and strips scripts. Confirm it is deployed (it is, at `2f3238f`). Identify the offending card row.

Containment: Edit the card row to remove the payload (content change, immediate). Query `cards.layers` for `<script`, `onerror=`, `onload=`, `javascript:`.

Recovery: Clean any other affected rows; review the ingest pipeline that admitted it.

Verify: Re-run the catalogue scan; sanitiser tests green.

Follow-up: Add ingest-time sanitisation/validation; accelerate CSP enforcement (§G).

8 · CSP rollout breakage

15 min: Symptom: legitimate resources blocked after enforcing the CSP. Identify the blocked directive from a report/console.

Containment: Roll back the CSP header to `Content-Security-Policy-Report-Only` (or revert the `vercel.json` commit) and redeploy. The report-only policy blocks nothing.

Recovery: Widen the policy for the legitimate origin (e.g. GeoGebra, fonts, Supabase), re-observe, then re-enforce.

Verify: Full session surface loads; no legitimate violations.

Follow-up: Keep the collector's privacy minimisation (path-only, no IP).

9 · Account-deletion failure / orphaned screenshots / incomplete anonymisation

15 min: Reproduce with a test account on staging. The Edge Function fails *loud* if the identity delete fails, returning a "contact support" error — check its logs.

Containment: If data was erased but the identity survived, that user is a flagged orphan — record the user id (no other PII).

Recovery APPROVAL**:** Manually complete deletion via the dashboard (delete the auth user; drain their screenshot folder). Confirm the report row is anonymised (null user_id/user_agent/ screenshot_path/viewport/metadata).

Verify: `test-user-data-lifecycle.mjs` on staging.

Follow-up: Fix the failure cause; the privacy policy promises deletion works.

10 · Domain / DNS / certificate outage

15 min: Confirm whether it is DNS resolution, the cert, or the registrar. The current live origin is `strata-nine-pi.vercel.app` (Vercel-managed cert); a custom-domain move adds registrar/cert steps.

Containment: If a recent DNS change broke it, revert the record. **Do not delete the Search Console TXT record.**

Recovery APPROVAL**:** Fix at the registrar; allow for TTL; re-issue/verify HTTPS; update auth Site URL + OAuth redirects if the origin changed.

Verify: HTTPS valid; site loads; auth redirects work.

11 · Service worker traps users on a broken shell

15 min: Symptom: users stuck on an old/broken UI after a deploy. Cause: a change to a cached shell file in `public/sw.js` without bumping the cache version.

Containment: Bump the `CACHE` version string in `sw.js`; the activate handler purges old caches on next load. Redeploy.

Recovery: The worker is network-first for navigation + calls skipWaiting, so most users recover on next launch.

Verify: A returning session gets the new shell.

Follow-up: Add "bump sw cache version" to the shell-change checklist.

12 · Lost admin access / primary operator unavailable

15 min: Identify which system's access is lost. Use the access matrix (§A) recovery paths.

Containment: Use provider account-recovery (2FA backup codes, recovery email). For a team resource, a second admin (if one exists) can restore access.

Recovery: Regain access; rotate anything that may have been exposed during recovery.

Follow-up APPROVAL**:** Add a second admin/owner where safe; store 2FA backup codes and the Android keystore offline. This is the biggest single-point-of-failure risk while Ali is the sole operator.

G · Security operations & CSP completion

Where the logs are

What	Where
Build / serverless function logs	Vercel dashboard → project → Deployments / Functions
Database / Auth / Edge Function logs	Supabase dashboard → Logs
Code history	GitHub — `origin/main`, commit messages, the agent board
Client errors / blocked resources	Browser dev tools → Console + Network
CSP violations	Once wired: the `/api/csp-report` collector's Vercel logs

Secret inventory & rotation order

Values never recorded here. Rotate in an order that avoids locking yourself out.

Secret	Stored in	Consumed by	Rotation note
Supabase service-role key	Supabase dashboard	`delete-account` secret; local ingest scripts	Highest impact. Rotate → update Edge Function secret → update local env. App keeps working (browser never uses it).
Supabase anon key	`.env.production` (public)	Browser	Public by design; rotating needs a client redeploy.
SMTP password	Supabase dashboard	Supabase Auth email	Rotate at provider → update in Supabase.
Google OAuth client secret	Supabase dashboard	Supabase Auth Google provider	Rotate in Cloud Console → update in Supabase.
Vercel / GitHub tokens	Provider account	CI/deploy	Rotate in provider settings.
Android upload/ signing key	Offline keystore	Play upload	Loss is near-unrecoverable — back up offline; Play App Signing key reset is the only remedy.

CSP: collector → observe → enforce

Current state: security headers live, CSP in **report-only**, collector **dormant** and unreferenced. F-02 is therefore latent, not active; the F-01 sanitiser is the working XSS defence. Completion sequence (design in `docs/engineering/CSP-REPORTING.md`):

1. **APPROVAL** Wire the collector: add `Reporting-Endpoints` header + `report-to` / `report-uri` directives pointing at `/api/csp-report`. Deploy. Confirm it 204s.
2. Observe 1–2 weeks across the full session surface (sign-in, reader, workshop, quiz, audio, Three.js, GeoGebra), desktop + Android.
3. Review reports (privacy-minimised: path-only, noise dropped, no IP). Widen the policy for every legitimate blocked origin.
4. **APPROVAL** Flip `Content-Security-Policy-Report-Only` → `Content-Security-Policy` in a separate commit. F-02 becomes active.
5. Rollback if anything legitimate breaks: revert to report-only and redeploy (Playbook 8).
6. Keep monitoring post-enforcement for regressions.

Enforcement criteria: a clean observation window with zero legitimate violations; every external origin accounted for; tested on real Android.

Current known risks / launch blockers

Must match `docs/PUBLIC-BETA-CHECKLIST.md`. As of 2026-07-22: CSP not yet enforcing; both DB tests not yet executed against staging; support-mailbox monitoring is an operational obligation; Play verification pending the Companies House certificate (external long-pole).

H · User support, privacy & emergency data requests

Self-service export & deletion

Users act from **Settings → Your data**. Export returns a JSON of all their rows. Deletion is immediate and irreversible: it **erases** login, profile, all progress/attempts/rewards/activity, and screenshots; it **anonymises** issue reports (fault kept, reporter stripped). This matches the published privacy policy.

Locked-out user

1. Prefer self-service: password reset via email.
2. If escalated: verify identity proportionately before any action; act least-privilege; keep an evidence record (what was requested, what was done, when).
3. Deliver any export securely to the account's own verified address.
4. Never expose a service key or disable a security control to "help".

Support mailbox & breach handling

- `admin@arcavetech.co.uk` is the published route for data/deletion requests — it must be monitored (owner to confirm, §A).
- Log requests and responses; apply a consistent retention decision.
- On a suspected personal-data breach, escalate to Ali. Do **not** invent statutory deadlines — follow approved legal advice; the ICO is the UK supervisory authority.

I · One-page emergency reference

Print this page on its own. Contains no secrets.

Production	`strata-nine-pi.vercel.app` · Supabase `wmetdmfsniqrshuaoodc` (London) · commit `2f3238f`
Staging	`qubix-staging.vercel.app` · Supabase `atmmfkhjsdqqwnhqifxm`
Retired — do not use	Supabase `xzesbcrlnbesmvxmgotp`

STOP before production

A Git push is not a deploy · production deploy, schema/RLS, DNS, secret rotation, destructive-data, and CSP-enforce all need Ali's approval · never disable RLS · never expose the service key · never run destructive tests off staging · never copy real users to staging.

Deploy	`pnpm run build:production && pnpm run deploy` → capture `dpl_` id → smoke test
Rollback (site)	Vercel → reassign stable alias to last good deployment ID
Rollback (content)	Edit the `cards` row back
Rollback (schema)	No auto-undo → corrective migration (staging-tested)
Security tests	`pnpm run test:security` (47/47)
Headers expected	CSP report-only · HSTS · X-Frame-Options DENY · nosniff · referrer strict-origin

Symptom	First check	Safe action	Escalate?
Site down	Vercel deploy status	Roll back alias	If not resolved
Can't sign in	Supabase Auth logs	Roll back recent client	SMTP/OAuth cfg
Email not arriving	SPF/DKIM + provider	—	Yes if domain-wide
Suspected key leak	Where it leaked	Preserve evidence	**Yes — rotate w/ Ali**
Cross-user data	Which table	Do NOT disable RLS	**Yes**
Bad card / XSS	cards.layers scan	Edit row (sanitiser holds)	If widespread
Old shell stuck	sw.js cache version	Bump + redeploy	No
Deletion failed	Edge Fn logs	Record orphan uid	Yes — complete manually

Deeper runbooks	This guide §C deploy · §D DB · §E disaster recovery · §F playbooks · §G CSP/secrets. Live status: `docs/PUBLIC-BETA-CHECKLIST.md`. Config: `docs/ENVIRONMENTS.md`, `docs/RELEASE-MODEL.md`.
Owners	Primary operator: Ali. Support mailbox: `admin@arcavetech.co.uk`. (Add a second admin — single-operator risk.)

Appendix · Corrections applied since guide v1.0

Codex's review of the first PDF found factual imprecisions. Each is corrected in this version:

v1.0 said	Corrected to
"a push/merge to main automatically updates staging"	**UNVERIFIED** — the intended flow is feature→staging→QA→main→manual prod deploy, but that `qubix-staging` follows `origin/staging` is **pending confirmation** in the Vercel dashboard (per the checklist). State only what is verified.
"database changes are not versioned by Git"	Migration files **are** version-controlled; *applying* them to a Supabase project is a separate manual action, effective immediately, not rolled back by Git or Vercel.
"no server of its own"	No always-on custom server, but Supabase Edge Functions and a dormant Vercel serverless CSP collector exist.
`npm run test:security`	`pnpm run test:security` (repo-standard).
"full schema" / "every API call" with only 2 tables detailed	Full schema/API appendix added (§B) covering every table and call site.
Android wording	Web/UI changes avoid a Play update; wrapper/app-ID/icon/permissions/target-URL/SDK/signing/Digital-Asset-Links changes need a new signed `.aab`.

> Generated from `docs/qubix-admin-guide.src.html` via `docs/build-admin-guide.py` (WeasyPrint). To regenerate after editing the source: `python3 docs/build-admin-guide.py`. Reflects `origin/main = origin/staging = 2f3238f`, 2026-07-22. The authoritative live-status list is `docs/PUBLIC-BETA-CHECKLIST.md`; where it conflicts with this guide, it wins.